UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERIA EN SISTEMAS DE INFORMACIÓN

SISTEMAS OPERATIVOS I

INGENIERO JULIO GÓMEZ

SECCIÓN “A”

Proyecto – Simulador Kernel

CRISTIAN JOSUÉ CRUZ LÓPEZ

Carné: 7690-24-7788

03/11/2026

iii. Resumen Ejecutivo

A continuación, veremos en el siguiente proyecto la construcción de un simulador de funciones del kernel de un sistema operativo y es que el presente proyecto nos mostrará la función de lo antes mencionado mediante una interfaz de línea de comandos escrita en el lenguaje de programación Python. Nuestro propósito principal al cual va enfocado este proyecto es de llevar a la práctica los conceptos teóricos aprendidos en el curso de SOI, modelando de manera lógica las funciones más importantes de un sistema operativo sin la necesidad de interactuar directamente con el hardware físico de la máquina.

Este simulador integra de manera modular cinco áreas fundamentales, las cuales son: representar y gestionar procesos mediante bloques de control de procesos, los famosos PCB, la administración estática y dinámica de la memoria RAM con un límite que se nos estableció el cual es de 100 MB y el manejo de errores por falta de memoria, dicha aplicación multitarea llega a utilizar el algoritmo de Round Robin con un quantum fijo el cual es de 3s, el control de concurrencia a través de un semáforo binario para poder garantizar exclusión mutua en una sección mutua en una sección de carácter crítico compartida, y la simulación, detección y resolución de interbloqueos o llamados en inglés los DeadLock mediante el aborto forzado de procesos involucrados en ciclos de espera circular.

Nuestro proyecto fue puesto a prueba mediante algunos casos de prueba que fueron controlados y que estos validaron el correcto cambio de contexto de la CPU, la protección del espacio de memoria y la resolución efectiva de conflictos de concurrencia y bloqueos mutuos. Toda la experiencia de desarrollo, las pruebas de estrés ejecutadas y las lecciones aprendidas se reumen al final.

iv. Índice

iii. Resumen Ejecutivo 2

iv. Índice 3

v. Introducción del problema a resolver 4

vi. Identificación del problema 4

vii. Justificación 5

viii. Objetivos (Estándar SMART) 5

Objetivo general 5

Objetivos Específicos 6

ix. Marco Teórico 7

1. Concepto de Kernel y sus funciones 7

2. Estructura de los Procesos (PCB vs. TCB) 7

3. Gestión de Memoria y Límites Físicos 7

4. Algoritmos de Planificación de CPU (Round Robin) 7

5. Concurrencia, Sección Crítica y Semáforos Binarios 8

6. Interbloqueos y Condiciones de Coffman 8

x. Metodología 9

Desarrollo y ejecución 9

xi. Declaración de Autoría y Honestidad Académica 12

xii. Conclusiones y Recomendaciones 13

Conclusiones 13

Recomendaciones 14

v. Introducción del problema a resolver

El poder entender cómo funciona un SO de manera interna, para nosotros los principiantes no siempre es tarea fácil cuando solo se estudia la parte teórica que se nos enseña en los libros.

Conceptos como la asignación de memoria RAM, la rotación de los turnos en el procesador o la prevención de colisiones cuando dos programas intentan usar el mismo recurso al mismo tiempo pueden sonar abstractos si no se observa su comportamiento en la práctica.

Para resolver este problema de aprendizaje, se propuso la creación de un programa simulador de Kernel. Este simulador actúa como un modelo simplificado pero fiel de núcleo de un SO. A través de este entorno interactivo en consola, es posible crear procesos simulados, asignarles un espacio de memoria, ver cómo el procesador se los turna en rodajas de tiempo de 3 segundos, observar cómo un semáforo bloquea a un proceso que quiere usar una variable ocupada y solucionar los bloqueos mutuos donde dos procesos se quedan congelados esperando recursos.

vi. Identificación del problema

En los entornos de computación reales, cuando la memoria física se agota o cuando dos tareas intentan escribir en el mismo sector de memoria de forma desordenada, ocurren fallos graves como congelamientos del sistema o cierres inesperados. De la misma manera, si el planificador de la CPU no distribuye bien los tiempos, un proceso largo podría monopolizar el procesador dejando sin responder a las demás aplicaciones.

El problema específico a resolver en este proyecto consiste en modelar en software las estructuras de datos y algoritmos de control capaces de:

1. Rechazar peticiones de creación de procesos cuando la RAM simulada de 100 MB se encuentre llena.

2. Evitar la inanición de procesos repartiendo la CPU de forma equitativa mediante el algoritmo Round Robin (q=3).
3. Proteger las variables globales compartidas impidiendo condiciones de carrera.
4. Alertar de forma oportuna cuando ocurra una condición de Deadlock, espera circular entre dos procesos, y brindar herramientas al operador para romper el ciclo.

vii. Justificación

El desarrollo de este proyecto se justifica desde dos perspectivas:

- **Educativa y Práctica:** Permite consolidar de forma clara los conocimientos teóricos del curso de Sistemas Operativos I. Escribir el código de un semáforo binario o de una cola de listos ayuda a comprender exactamente qué ocurre detrás de escena en sistemas operativos reales como Linux o Windows.
- **Técnica y Modular:** Demuestra cómo con un lenguaje claro como Python es posible estructurar mediante programación orientada a objetos, una arquitectura limpia, legible y mantenible, utilizando listas y colas para representar componentes complejos del Kernel sin recargar el sistema informático.

viii. Objetivos (Estándar SMART)

Objetivo general

El objetivo principal de este proyecto es el de poder diseñar y desarrollar un simulador funcional del kernel de un sistema operativo en consola usando el lenguaje de programación Python para la fecha 3 de noviembre del presente año, logrando cumplir en la medida de lo posible con las especificaciones de gestión de memoria de una capacidad de 100 MB,

planificación Round Robin, concurrencia mediante semáforos binarios y resolución de DeadLocks.

Objetivos Específicos

1. **Memoria:**

   Programar un módulo de gestión de memoria física que valide el límite estático de 100 MB en menos de 0.1 seg por petición, emitiendo un mensaje de error Out of Memory cuando este exceda la capacidad disponible de memoria.

2. **Planificación:**

   Implementar la cola de listos con el algoritmo Round Robin asignado un Quantum exacto de 3 segundos por turno para garantizar que todos los procesos ingresados puedan percibir tiempo de CPU dentro de las pruebas del prototipo,

3. **Sincronización:**

   Modelar las funciones **wait ()** y **signal ()** de un semáforo binario para asegurar que únicamente 1 proceso pueda estar dentro de la sección crítica simulada a la vez.

4. **DeadLock:**

   Este consiste en crear una función capaz de simular la condición de espera circular entre dos procesos y proveer una opción CLI para romper el interbloqueo abortando el proceso víctima seleccionado por el usuario.

ix. Marco Teórico

1. Concepto de Kernel y sus funciones

El Kernel o núcleo es la parte principal del sistema operativo. Es el software responsable de administrar los recursos del sistema (procesador, memoria RAM, dispositivos de entrada y salida) y servir de puente entre las aplicaciones del usuario y el hardware real.

2. Estructura de los Procesos (PCB vs. TCB)

- **PCB (Process Control Block):** Es la estructura de datos en memoria donde el sistema operativo guarda la información de un proceso: su identificador, estado (NUEVO, LISTO, EJECUCIÓN, BLOQUEADO, TERMINADO), memoria asignada y tiempos de CPU.
- **TCB (Thread Control Block):** Representa un hilo individual de ejecución dentro de un proceso. Mientras que el PCB maneja recursos globales y espacio de dirección de memoria, el TCB gestiona datos más ligeros específicos del hilo, como su propio contador de programa y registros de trabajo.

3. Gestión de Memoria y Límites Físicos

La memoria RAM es un recurso finito. El gestor de memoria se encarga de asignar espacio contiguo a los procesos cuando inician y de liberar dicho espacio cuando el proceso finaliza. Si la suma de la memoria usada más la memoria requerida supera el total disponible (100 MB en nuestra simulación), el Kernel debe rechazar la tarea mediante una alerta de desbordamiento.

4. Algoritmos de Planificación de CPU (Round Robin)

El algoritmo Round Robin es una técnica de planificación pensada para sistemas de tiempo compartido. Asigna a cada proceso un turno fijo en la CPU llamado Quantum representado con la

letra q. Si el proceso no termina durante su rodaja de tiempo, sufre un cambio de contexto y pasa al final de la cola de listos para permitir que el siguiente proceso ejecute. Esto garantiza equidad y evita que un programa se quede con el procesador indefinidamente.

5. Concurrencia, Sección Crítica y Semáforos Binarios

Cuando dos o más procesos intentan modificar la misma variable o archivo al mismo tiempo, ocurre una condición de carrera. Para evitarlo, la zona del código que manipula datos compartidos se llama Sección Crítica. Un semáforo binario actúa como una cerradura con dos estados (1 = Libre, 0 = Ocupado):

- wait(): Si el semáforo está libre, lo cambia a 0 e ingresa. Si está en 0, el proceso se bloquea y espera en cola.
- signal(): Al salir, si hay procesos esperando, le cede el turno al primero; si no hay nadie, cambia el semáforo a 1.

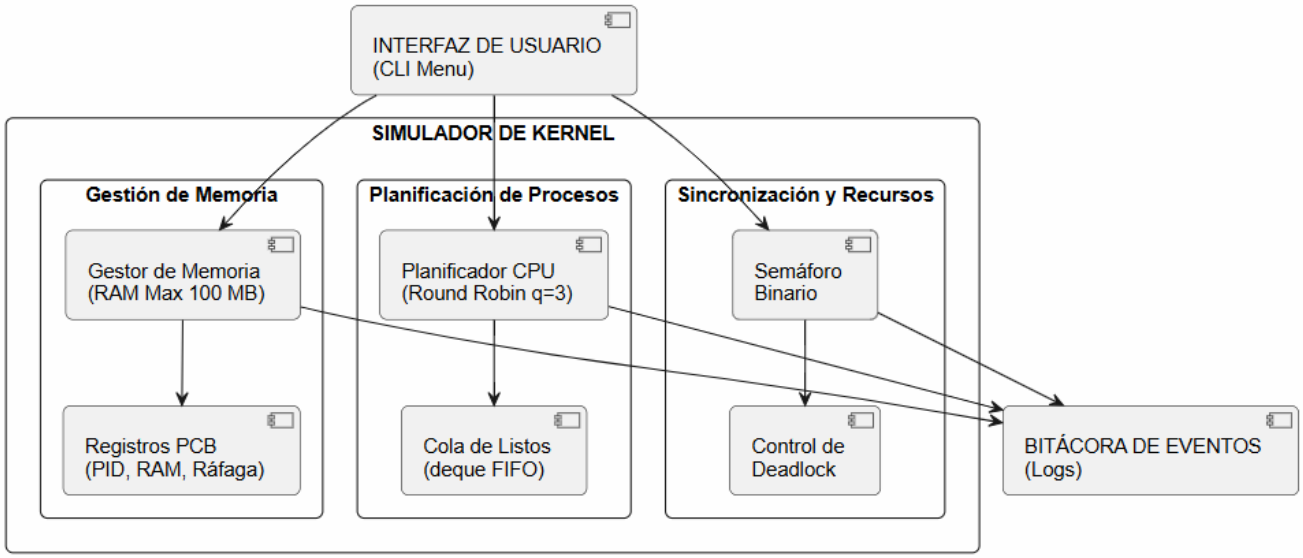
6. Interbloqueos y Condiciones de Coffman

Un Deadlock o interbloqueo es una situación en la que dos o más procesos quedan bloqueados para siempre porque cada uno tiene un recurso que el otro necesita. Ocurre cuando se cumplen 4 condiciones conocidas como las condiciones de Coffman: exclusión mutua, retención y espera, no apropiación y espera circular. En nuestro simulador, la técnica aplicada para resolverlo es la eliminación de la espera circular mediante el aborto selectivo de uno de los procesos involucrados.

x. Metodología

1. Diagnóstico y Diseño

A continuación, veremos la arquitectura lógica de nuestro simulador que se organizó en bloques modulares claramente definidos:



Desarrollo y ejecución

a. Stack tecnológico y versiones

- **Lenguaje de programación:** En este caso utilizamos el lenguaje de programación Python en su versión v3.10+
- **Entorno de Desarrollo (IDE):** Usamos el programa de Visual Studio Code v1.90+
- **Módulos Nativos Utilizados:** Se usó el comando **collections.deque,** este comando nos sirve para la gestión de colas FIFO, y el comando **time,** nos sirve para estampado de la fecha y hora de los logs que se ingresen.
- **Control de versiones:** Git y repositorio público en el software de GitHub.

b. Requerimientos específicos

A continuación, veremos los requerimientos más esenciales:

- Una interfaz 100% interactiva con el usuario a través de la consola, la CLI.
- Como las instrucciones lo mencionan, tener un límite de memoria RAM asignado de 100 MB.
- Asignación de un Quantum de CPU fijo a 3 segundos (q = 3).
- Una exclusión mutua mediante semáforo binario con métodos **wait ()** y **signal ()**.
- Detección explícita de DeadLock por la espera circular que esta tiene, con capacidad de abortar procesos.

c. Construcción del prototipo

El prototipo se organizó dentro de un único archivo en el software de Visual Studio Code, utilizando el lenguaje de programación de Python como ya se mencionó anteriormente, este archivo es reutilizable y está llamado como **kernel_simulator.py.** Este se diseñó para que la función auxiliar **print_red_box()** encierre todas las respuestas operativas de la consola dentro de marcos rojos decorados con texto amarillo encontrado en su interior, garantizando un aspecto limpio, estéticamente hablando también, y visualmente ordenado a la hora de poder llevar a cabo las demostraciones. Por eso se hizo de esta forma la construcción del prototipo para que sea considerado agradable ante la vista de quienes lo estén utilizando.

d. Pruebas del prototipo

1. **Prueba No. 1: Rechazo de Proceso por falta de RAM:**
    - **Procedimiento:** ingresamos un proceso 1 de 60 MB y luego intentamos ingresar un proceso 2 de 50 MB.
    - **Resultado:** como sabemos en el proyecto, la suma que nos da es de 110 MB, la cual excede los 100 MB de límite que le establecimos. Por lo cual el sistema rechazó inmediatamente al proceso 2 indicando que solo quedaban 40 MB de memoria disponibles.
2. **Prueba No. 2: Sucesión de Ráfagas en Round Robin con un quantum = 3:**
    - **Procedimiento:** Se ingresaron dos procesos con ráfagas de 5s y 2s respectivamente.
    - **Resultado:** El proceso 1 se ejecutó 3 seg, y este sufrió cambio de contexto y regresó a la cola con 2 seg restantes. Por lo tanto, el proceso 2 ejecutó sus 2 seg completos, finalizó y liberó su memoria. Ahora, finalmente, el proceso 1 retomó la CPU y concluyó sus últimos 2 seg.
3. **Prueba No. 3: Efectividad del Semáforo ante Condiciones de Carrera**
    - **Procedimiento:** Se ejecutó la prueba de concurrencia donde el Proceso 901 realiza **wait()** y, antes de liberar la sección crítica, el Proceso 902 realiza **wait().**
    - **Resultado:** El Proceso 901 ingresó exitosamente. El Proceso 902 fue enviado al estado **BLOQUEADO** dentro de la cola del semáforo. Al ejecutar **signal(),** el Proceso 902 fue desbloqueado y volvió a la cola de listos.

4. **Prueba 4: Detección y Eliminación de Deadlock**
    - **Procedimiento:** Se forzó la condición de espera circular entre los procesos PID 101 y PID 102.
    - **Resultado:** El Kernel emitió una alerta roja de interbloqueo. Al seleccionar la opción de resolución e ingresar el PID 101 como víctima, el proceso fue abortado, liberando sus recursos y permitiendo al Proceso 102 continuar.

xi. Declaración de Autoría y Honestidad Académica

A quien corresponda:

Por medio de la presente declaración, yo, **Cristian Josué Cruz López,** estudiante de la carrera de Ingeniería en Sistemas de Información de la Universidad Mariano Gálvez de Guatemala (UMG), identifico con el carné número **7690-24-7788,** por lo que certifico y declaro formalmente que:

1. El trabajo práctico y el código fuente presentados en este proyecto final para el curso de Sistemas Operativos I son de mi entera autoría y han sido desarrollados por mi persona de acuerdo con las instrucciones de la guía que se nos dejó, conforme lo recalcó el Ingeniero Julio también.
2. No he incurrido en conductas de plagio, copia total o parcial de proyectos de otros estudiantes o fuentes no citadas.
3. Las referencias teóricas tomadas de autores externos han sido debidamente indicadas en la sección de bibliografía.

Firmado a los 03 días del mes de noviembre del año 2026 en la ciudad de Guatemala Boca del monte.

Firma: 

xii. Conclusiones y Recomendaciones

Conclusiones

1. a implementación del simulador demostró prácticamente cómo la abstracción de estructuras de datos (PCB, colas de listos y tablas de memoria) permite modelar con precisión las decisiones lógicas de un Kernel sin necesidad de interactuar con hardware real.
2. El algoritmo Round Robin con q=3 demostró ser una solución eficaz para distribuir el tiempo de CPU equitativamente, confirmando que la elección del tamaño del Quantum es clave para evitar la sobrecarga por cambios de contexto innecesarios.
3. El uso de semáforos binarios y gestores de Deadlock es imprescindible en sistemas multitarea, ya que garantiza que los datos compartidos no sufran corrupción por condiciones de carrera y provee mecanismos para recuperar la operatividad ante bloqueos mutuos.

Recomendaciones

1. **Para futuras mejoras del proyecto:** Ampliar el módulo de gestión de memoria integrando simulación de paginación o memoria virtual con reemplazo de páginas, algoritmo FIFO o LRU.
2. **Para la ejecución de pruebas:** Mantener monitoreada la bitácora de eventos (logs) del sistema al realizar pruebas de carga para verificar el orden exacto en que ocurren las liberaciones de memoria y los cambios de estado.